AI Document Classification & Routing System — Project Documentation

Oct 17, 2026

1. Project Proposal

The project will deliver an AI service that reads an incoming document, assigns it to one of six categories and recommends where to route it, so employees only review the uncertain cases. Planned duration is 12 October to 4 December 2026, including one month of implementation (7-30 November), with a team of six.

Problem statement

The organization receives applications, complaints, invoices, contracts, requests and general correspondence through several channels. Today an employee opens each document, decides what it is and forwards it to the right department. This is slow, inconsistent between employees, and delays time-sensitive items such as complaints and invoices.

Objectives

1. Build a labeled dataset of at least 3,000 documents, with at least 500 per category.
2. Train a TF-IDF + classical ML baseline that reaches a macro F1 of 0.85 or higher.
3. Train an advanced NLP model (LSTM and a Hugging Face Transformer) that reaches a macro F1 of 0.92 or higher and beats the baseline by at least 3 points.
4. Produce a routing recommendation with a confidence score: auto-route above a confidence threshold, send the rest to human review.
5. Track every experiment in MLflow and package the selected model as an inference service with a live demo.

The numeric targets above are provisional: they are reviewed after the baseline results (M4, 13 Nov), and the final values are recorded in the evaluation report. The non-functional requirements (Section 7.4) use the same targets.

Document categories and routing map

Category	Typical content	Recommended route
Application	Job, service, permit or membership applications	HR / Admissions
Complaint	Customer or employee grievances,	Customer Service

Category	Typical content	Recommended route
	service failures	
Invoice	Bills, amounts due, tax and payment details	Finance (Accounts Payable)
Contract	Agreements, terms and conditions, amendments	Legal
Request	Requests for information, service or changes	Operations service desk
Correspondence	General letters, notices, FYI messages	Records / General Administration

Scope

In scope

- Labeled document dataset and a documented labeling guideline
- Text preprocessing pipeline (cleaning, normalization, tokenization), shared by training and inference
- TF-IDF baseline with linear classifiers (Logistic Regression, Linear SVM, Naive Bayes) and ensemble classifiers (Random Forest, XGBoost)
- Deep learning comparison: a BiLSTM model and a fine-tuned Hugging Face Transformer (e.g. DistilBERT or BERT-base)
- Class imbalance and data augmentation experiments
- Confidence calibration, routing threshold tuning and model compression for inference
- Evaluation with precision, recall and F1 (per class and macro), confusion matrix
- Error analysis of difficult and misclassified documents
- Experiment tracking and model registry in MLflow
- Packaged inference service (REST API in a Docker image) and an inference demonstration
- Technical architecture documentation

Optional extension

- Deploy the model to an Azure ML managed online endpoint
- Simple document-upload web interface that extracts text from text-based PDF and DOCX files, sends it to the endpoint and shows category, confidence and route

Out of scope

- OCR of scanned documents in the core version: the core system takes document text as input, and source datasets such as RVL-CDIP already provide OCR text

- Multi-label classification: each document gets exactly one of the six categories
- Integration with production email, ERP or ticketing systems
- Languages other than English in version 1
- Production-grade SLA, security accreditation and 24/7 support

Technical approach

1. **Data:** combine public corpora (see Data sources below) with synthetic samples generated per category and reviewed by a team member; synthetic samples are used for training only, never in the test set. Split 70/15/15 (train/validation/test), stratified by category.
2. **Baseline:** TF-IDF (word and character n-grams) feeding linear and ensemble classifiers, tuned with cross-validation; imbalance handling compared on top.
3. **Advanced:** BiLSTM with pretrained word embeddings, then a fine-tuned Transformer from the Hugging Face Hub; the best model on the validation set is promoted.
4. **Routing:** confidence scores are calibrated on the validation set; a configurable rules table maps each category to a department, and predictions below the confidence threshold go to human review.
5. **MLOps:** every run logs parameters, metrics, artifacts and the model to MLflow; the promoted model is versioned in the registry and served from there.

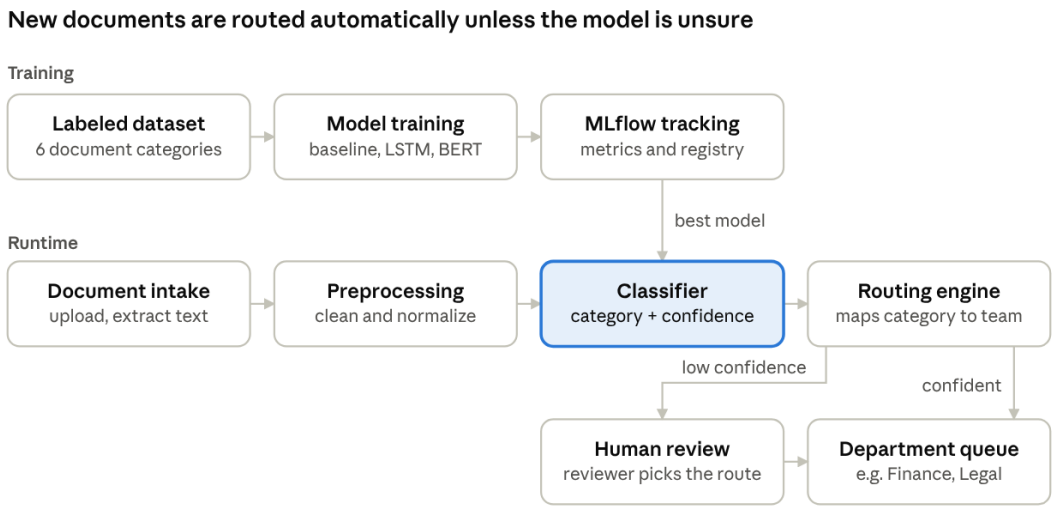
Data sources

No single public dataset covers all six categories, so the dataset combines several public sources. Numbers in brackets refer to Section 8.

Category	Main source	Notes
Application	RVL-CDIP "resume" class [10] + reviewed synthetic applications	Weakest category; synthetic samples stay out of the test set
Complaint	CFPB consumer complaint narratives [6]	Real narratives, personal information removed by the publisher
Contract	CUAD commercial contracts [11]	510 contracts across 25 contract types
Invoice	RVL-CDIP "invoice" class [10]; synthetic invoices [19]	RVL-CDIP OCR text is noisy
Request	Enron emails labeled by hand [14]; Bitext customer requests as seeds [2]	Bitext items are short messages, used for training only
Corresponden	RVL-CDIP "letter" and "memo"	Two real sources reduce

Category	Main source	Notes
ce	classes [10]; Enron emails [14]	source-style leakage

Solution architecture



Solution architecture · training flow and runtime routing flow

The same preprocessing module is used in training and at runtime. The classifier is served as a REST API (optionally an Azure ML endpoint), and documents below the confidence threshold go to a human reviewer instead of being auto-routed.

Deliverables

Deliverable	Description	Due
Planning documents	Proposal, plan, roles, risk register, KPIs	16 Oct (M1)
Literature review	Survey of document classification and routing approaches	16 Oct (M1)
Requirements specification	Categories, routing map, functional and non-functional requirements	16 Oct (M1)
Analysis and design document	Use cases, data flow, architecture, API and dataset design	6 Nov (M2)
Dataset	Labeled, split dataset + labeling guideline + data card	10 Nov (M3)

Deliverable	Description	Due
Preprocessing pipeline	Reusable Python module shared by training and inference	10 Nov (M3)
Baseline model	TF-IDF linear and ensemble models, logged in MLflow	13 Nov (M4)
Advanced NLP model	BiLSTM and fine-tuned Transformer, best one registered	23 Nov (M5)
Evaluation report	Per-class and macro precision, recall, F1; model comparison	26 Nov
Error analysis	Review of misclassified and low-confidence documents with causes	26 Nov
MLflow tracking	Tracking server, all experiments, model registry	Live from 10 Nov, ongoing
Inference demonstration	Dockerized REST API + live demo	30 Nov (M6)
Technical architecture	Architecture document and diagrams	30 Nov (M6)
Optional: Azure ML + UI	Managed endpoint and upload interface	28 Nov
Final presentation, test report and final reports	Testing results, final report, slides	4 Dec (M7)

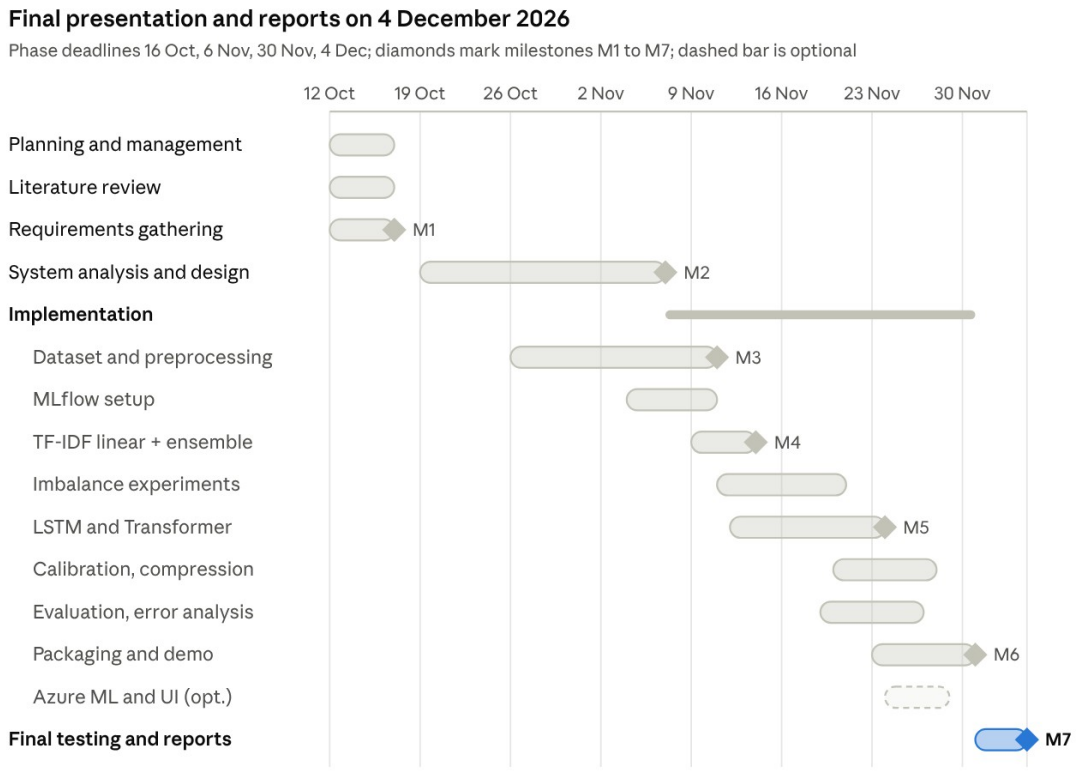
Assumptions and open questions

- Graduation project for an ML training program: team of six with equal workload. The project's direct stakeholders are the team and the lecturer/evaluators; the stakeholder analysis (Section 7.1) describes the intended users of the system in a realistic deployment.
- No real organization data: all training and test documents come from public sources plus reviewed synthetic samples.
- Phase deadlines 16 Oct, 6 Nov, 30 Nov and 4 Dec 2026; the first three phases are assumed to start on 12 October.
- Input is English document text; text extraction from PDF and DOCX files belongs to the optional upload interface.
- Dataset preparation starts during design (26 Oct) so implementation can start modeling immediately on 7 November.
- Open question: is an Azure subscription with Azure ML available for the optional extension?

2. Project Plan

The work follows six planned phases with fixed deadlines on 16 October, 6 November, 30 November and 4 December 2026, the last being the final presentation, testing and reports. Implementation is split into eight sub-tasks (5a-5h) with milestones every 3-7 days.

Timeline (Gantt chart)



Project schedule · 6 phases, 8 implementation sub-tasks, 7 milestones, 12 Oct to 4 Dec 2026

Dataset preparation starts during design (26 Oct) so labeled data is ready by 10 November; the start date of 12 October for the first three phases is assumed.

Phases and key tasks

Phase	Dates	Key tasks	Output
1. Project planning and management	12-16 Oct (continues to 4 Dec)	Proposal, plan, roles, risk register, KPIs; weekly tracking afterwards	Planning documents
2. Literature	12-16 Oct	Each member reviews the methods	Literature

Phase	Dates	Key tasks	Output
review		they own: TF-IDF + linear and ensemble models, LSTM, Transformers, imbalance handling, calibration	review
3. Requirements gathering	12-16 Oct	Categories, routing map, functional and non-functional requirements, KPI targets	Requirements specification
4. System analysis and design	19 Oct-6 Nov	Use cases, data flow, system architecture, evaluation protocol, dataset design and labeling guideline	Analysis and design document
5. Implementation (source code and execution)	7-30 Nov	Sub-tasks below	Source code, models, demo
5a. Dataset and preprocessing	26 Oct-10 Nov	Collect and label data, audit labels, cleaning, tokenization	Labeled dataset + preprocessing pipeline
5b. MLflow setup	4-10 Nov	Tracking server, experiment naming, model registry	Working MLflow server
5c. TF-IDF linear and ensemble models	9-13 Nov	LR / SVM / NB and Random Forest / XGBoost, cross-validated tuning	Baseline models + results
5d. Imbalance and augmentation experiments	11-20 Nov	Class weights, oversampling, synthetic samples, tested on the baselines and the LSTM	Imbalance study
5e. LSTM and Transformer	12-23 Nov	BiLSTM; fine-tune Hugging Face Transformer on GPU; hyperparameter search	Best advanced model registered
5f. Calibration and compression	20-27 Nov	Confidence calibration, routing threshold tuning, ONNX export or quantization, latency tests	Calibrated, compressed routing model
5g. Evaluation and error	19-26	Test-set metrics for all models, confusion matrices, review of	Evaluation + error

Phase	Dates	Key tasks	Output
analysis	Nov	misclassified docs	analysis results
5h. Packaging and inference demo	23-30 Nov	Inference service with routing rules, Docker image; optional Azure ML endpoint and upload UI with PDF/DOCX text extraction (24-28 Nov)	Inference demo
6. Final presentation, testing and reports	1-4 Dec	End-to-end testing, user test, final reports, presentation	Final delivery

Milestones

ID	Milestone	Target date	Acceptance criterion
M1	Planning, literature review and requirements delivered	16 Oct 2026	Planning documents, literature review and requirements approved
M2	Analysis and design approved	6 Nov 2026	Design document and architecture reviewed
M3	Dataset v1 frozen	10 Nov 2026	3,000+ labeled docs, label agreement 90%+ on audit sample
M4	Baseline results	13 Nov 2026	Baseline macro F1 logged in MLflow
M5	Advanced model selected	23 Nov 2026	Best model registered, beats baseline on validation set
M6	Implementation complete	30 Nov 2026	Evaluation and error analysis done; API classifies and routes an uploaded document end to end
M7	Final presentation, testing and reports	4 Dec 2026	All deliverables accepted

Resource allocation

All six members carry an equal share of the work in every phase; the table shows each member's main focus per phase.

Member	Planning, review, requirements (12-16 Oct)	Analysis and design (19 Oct-6 Nov)	Implementation (7-30 Nov)	Final testing and reports (1-4 Dec)
Member 1	Review of imbalance and augmentation methods; category requirements	Dataset design, labeling guideline; labeling from 26 Oct	Dataset freeze, imbalance and augmentation experiments	Data and imbalance sections, slides
Member 2	Review of TF-IDF and linear classifiers	Preprocessing design; labeling	Preprocessing pipeline, linear models	Linear model results, slides
Member 3	Review of ensemble methods and evaluation metrics	Evaluation protocol design; labeling	Ensemble models, evaluation report	Evaluation section, slides
Member 4	Review of LSTM approaches and MLflow	LSTM and experiment tracking design; labeling	MLflow setup, LSTM model	LSTM results, slides
Member 5	Review of Transformer approaches	Transformer and GPU plan; labeling	Transformer fine-tuning, error analysis	Error analysis section, slides
Member 6	Review of calibration, thresholds and model serving	Routing logic and serving design; labeling	Calibration, compression, inference demo	Architecture section, live demo

Tools and infrastructure

Resource	Use	Notes
Python, scikit-learn, PyTorch, Hugging Face Transformers	Modeling	Open source
GPU (Kaggle / Colab, or Azure ML compute)	LSTM and Transformer training	Book GPU time for 12-23 Nov
MLflow server	Experiment tracking and model registry	Local server or Azure ML workspace
Git + GitHub, project board	Code, reviews, task	One repo, protected main

Resource	Use	Notes
	tracking	branch
FastAPI + Docker	Inference service	Used for the demo and Azure deployment
Azure ML (optional)	Managed online endpoint	Needs an Azure subscription

3. Task Assignment and Roles

All six members carry an equal workload with no team lead, and every member owns a machine learning experiment: each trains, evaluates and logs their own models in MLflow, plus one supporting task. Planning, design, documentation and the final presentation are shared by everyone.

Roles and responsibilities

Member	ML work owned	Supporting task	Owns
Member 1	Class imbalance and data augmentation experiments: class weights, oversampling, synthetic samples; effect on per-class F1	Data sourcing, labeling guideline, label audit, dataset splits	Dataset, data card, imbalance study
Member 2	TF-IDF + linear models: Logistic Regression, Linear SVM, Naive Bayes; n-gram and feature settings	Preprocessing pipeline (cleaning, normalization, tokenization)	Preprocessing pipeline, linear baseline
Member 3	TF-IDF + ensemble models: Random Forest, XGBoost; feature selection	Shared evaluation scripts, confusion matrices, model comparison	Ensemble models, evaluation report
Member 4	BiLSTM with pretrained word embeddings; sequence length and architecture tuning	MLflow tracking server and model registry (everyone logs their own runs)	LSTM model, MLflow tracking
Member 5	Hugging Face Transformer fine-tuning (DistilBERT / BERT); hyperparameter search	Error analysis of misclassified and low-confidence documents	Transformer model, error analysis
Member 6	Confidence calibration and routing threshold tuning; model compression (ONNX export or	Inference service, demo, optional Azure ML deployment and	Calibrated routing model,

Member	ML work owned	Supporting task	Owns
	quantization) and latency tests	upload UI	inference demo, technical architecture

Shared by all six members: project planning documents, literature review (each member covers the methods they own), requirements, system analysis and design, end-to-end testing, final report and presentation. Coordination (running the stand-up, updating the board and the risk register) rotates weekly between members.

Between 26 October and 10 November every member spends some time labeling so the dataset is frozen by 10 November; this keeps the one-month implementation phase for modeling, evaluation and packaging.

Working agreements

- Daily 15-minute stand-up plus a weekly 45-minute review on Thursday.
- Every change goes through a pull request reviewed by one other member.
- Every training run is logged to MLflow; results not in MLflow do not count for model selection.
- Each milestone ends with a short demo to the whole team before it is marked done.

4. Risk Assessment and Mitigation Plan

The highest risks are schedule slip in the one-month implementation window and too little realistic labeled data; both are mitigated by parallel work streams and a fallback model that is always ready to ship.

Scoring: Likelihood and Impact on a 1-3 scale (1 = low, 3 = high); Score = Likelihood x Impact. Risks scoring 6 or more are reviewed at every stand-up.

ID	Risk	Score (L x I)	Mitigation (prevent)	Contingency (if it happens)	Owner
R1	Not enough real labeled documents; dataset late for M3	9 (3 x 3)	All six members label between 26 Oct and 10 Nov; combine public corpora with reviewed synthetic samples; labeling guideline on day 1	Freeze a smaller v1 (min. 300 per class) on time and grow it while the baseline trains	Member 1
R2	Schedule slip against the	9 (3 x 3)	Parallel streams from day one; daily	Drop the optional Azure	All (weekly coordinato

ID	Risk	Score (L x I)	Mitigation (prevent)	Contingency (if it happens)	Owner
	fixed phase deadlines (16 Oct, 6 Nov, 30 Nov, 4 Dec)		stand-up; milestones every 2-4 days; scope cut list agreed at M1	ML extension first, then the LSTM tuning; never the baseline or the demo	r)
R3	Synthetic data does not reflect real documents, so test scores are too optimistic	6 (2 x 3)	Keep the test set real (public sources only); never generate test data	Report scores on real and synthetic test sets separately	Member 1
R4	Overlapping categories (e.g. Request vs Complaint, Application vs Request) confuse labelers and models	6 (3 x 2)	Clear definitions with examples per class; audit 10% of labels; measure label agreement	Merge or redefine classes at M3; route ambiguous cases to human review	Member 5
R5	Source leakage: the model learns which dataset a document came from instead of its category	6 (3 x 2)	Strip source markers (headers, boilerplate) in preprocessing; use two real sources per category where possible	Analyze errors per source in the error analysis and report it as a limitation	Member 5
R6	GPU time unavailable or too slow for Transformer fine-tuning	6 (2 x 3)	Book Kaggle/Colab/Azure GPU early; start with DistilBERT; cap sequence length at 512 tokens	Ship the best classical or LSTM model; the Transformer becomes a stretch goal	Member 5
R7	Class imbalance (e.g. few contracts or applications)	4 (2 x 2)	Stratified splits, class weights, targeted collection	Lower that class's routing threshold so more go to	Member 1

ID	Risk	Score (L x I)	Mitigation (prevent)	Contingency (if it happens)	Owner
	lowers recall on minority classes		for small classes	human review	
R8	Noisy OCR text in source datasets (e.g. RVL-CDIP) lowers accuracy and distorts comparisons	4 (2 x 2)	Text cleaning in preprocessing; drop near-empty samples; track scores per source	Report noisy-source results separately; exclude the noisiest source from the test set	Member 2
R9	Transformer too slow or too large for the inference latency target	4 (2 x 2)	Measure latency by 23 Nov; use DistilBERT, ONNX export or quantization	Serve the faster baseline in the demo and document the trade-off	Member 6
R10	Experiments not reproducible or not logged	4 (2 x 2)	MLflow live from 10 Nov; fixed random seeds; data versioned by hash	Rerun the final candidate models from logged parameters before M5	Member 4
R11	Team member unavailable (illness, exams)	4 (2 x 2)	Every deliverable has a backup reviewer; code reviewed and documented as it is written	Backup takes over; the team re-plans scope at the next stand-up	All (weekly coordinator)
R12	Personal details in public source documents leak into logs or the repository	3 (1 x 3)	Anonymize before labeling; private repo and storage; no raw text in logs or MLflow artifacts	Remove affected data and rotate access	Member 1
R13	Azure subscription, quota or cost issues block the optional deployment	2 (2 x 1)	Confirm access during design; set a budget alert; delete endpoints after the demo	Demo the Dockerized API locally instead	Member 6

Scope cut order

If the schedule slips, scope is cut in this order: (1) Azure ML deployment and upload UI, (2) LSTM hyperparameter search, (3) Transformer size (fall back to DistilBERT). The dataset, baseline, evaluation, error analysis, MLflow tracking and inference demo are never cut.

5. Key Performance Indicators (KPIs)

The project succeeds if the advanced model reaches a macro F1 of 0.92 on the real test set, routes at least 80% of documents automatically with 95%+ routing accuracy, and answers in under 500 ms (p95). Targets are provisional: they are reviewed after the baseline results (M4, 13 Nov), and the final values are recorded in the evaluation report.

Model quality

KPI	Definition	Target	Measured
Macro F1 (advanced model)	Unweighted mean F1 across the 6 categories, real test set	0.92 or higher	At M5 and in the evaluation (26 Nov), logged in MLflow
Macro F1 (baseline)	Same, TF-IDF + classical model	0.85 or higher	At M4
Improvement over baseline	Advanced macro F1 minus baseline macro F1	+3 points or more	Evaluation (26 Nov)
Per-class recall	Recall for each category	0.85 or higher for every class	Evaluation (26 Nov)
Per-class precision	Precision for each category	0.85 or higher for every class	Evaluation (26 Nov)
Label quality	Agreement between two labelers on the 10% audit sample	90% or higher	At M3

Routing effectiveness

KPI	Definition	Target	Measured
Automation rate	Share of documents above the confidence threshold, routed with no human step	80% or higher	Test set at M6
Routing	Share of auto-routed documents sent to	95% or	Test set at

KPI	Definition	Target	Measured
accuracy (auto-routed)	the correct department	higher	M6
Human review rate	Share of documents sent to human review because confidence is low	20% or lower	Test set at M6
Triage time per document	Time from upload to routing decision, vs. a timed manual triage of the same documents	Under 1 minute (manual timed in the user test)	User test during final testing

System performance

KPI	Definition	Target	Measured
Response time	p95 latency of one classification request (text up to 512 tokens)	Under 500 ms on CPU; under 200 ms for the baseline	Load test at M6
Throughput	Documents classified per minute by one service instance	100 or more	Load test at M6
System uptime	Share of time the endpoint answers health checks	99% or higher during final testing and the demo	Azure ML / container monitoring
Error rate	Share of requests that fail with a server error	Under 1%	Service logs

User adoption (user test during final testing, 1-4 Dec)

KPI	Definition	Target	Measured
User adoption rate	Share of test users (e.g. 10 trainees) who would choose the system over manual sorting	70% or higher	Short survey after the user test
Task success rate	Share of test users who upload a document and read its route without help	90% or higher	Observed during the user test
User satisfaction	Average rating of routing suggestions (1-5 scale)	4.0 or higher	Survey after the user test

Project delivery

KPI	Definition	Target	Measured
Phase deadlines met	Deadlines on 16 Oct, 6 Nov, 30 Nov and 4 Dec met	4 of 4	At each deadline
Milestones on time	Milestones M1-M7 met on their target date	6 of 7 or more	At each milestone
Experiment tracking coverage	Training runs logged in MLflow with params, metrics and model	100%	At M5
Deliverables accepted	Required deliverables accepted at final review	14 of 14	At M7 (4 Dec)

6. Literature Review

Existing tools show that classification and routing can be automated, but they are either managed cloud services or single research models; this project builds and compares several of its own models inside one routing workflow. Numbers in brackets refer to Section 8.

6.1 Introduction

Document classification and routing are established applications of Artificial Intelligence (AI) and Natural Language Processing (NLP). Organizations receive applications, complaints, invoices, contracts, requests and correspondence, and sorting them by hand takes time, especially when employees must both identify the document type and choose the right department.

This review first examines three relevant solutions: Google Cloud Document AI, Microsoft Azure AI Content Understanding, and DocBERT. It then reviews the methods each team member applies in the project, compares the approaches, and identifies the gap the project addresses.

6.2 Review of similar solutions

6.2.1 Google Cloud Document AI

Overview. Google Cloud Document AI provides document processing capabilities, including custom document classification. Its Custom Classifier identifies document types from predefined categories and can be adapted to a specific document collection through training. It supports workflows in which documents must be classified before being passed to other processing components [8].

Advantages

- Supports custom document categories based on organizational requirements.
- Provides pretrained classification capabilities with options for customization.
- Integrates with other document processing components.
- Automates the identification of document types, reducing manual classification.

Limitations

- Requires familiarity with Google Cloud and its configuration.
- Usage may involve cloud service costs, depending on the model and volume.
- Relies on an external cloud platform instead of models the team builds and trains.
- Suitability depends on the document types, data quality and classification requirements.

Relevance to the proposed system. Document AI shows how custom classification is applied in real workflows. The project adopts the idea of predefined categories while implementing and evaluating its own models, so the team can compare approaches on the selected dataset.

6.2.2 Microsoft Azure AI Content Understanding

Overview. Azure AI Content Understanding classifies documents and routes them to processing components. Developers define custom categories and routing rules that send each classified document to a specific analyzer, which can then extract information from it. For example, an invoice is classified as an invoice and sent to a component that extracts invoice fields [15].

Advantages

- Supports both classification and routing workflows.
- Lets developers define custom categories and routing rules.
- Connects classification with further analysis and information extraction.
- Provides APIs and development tools for integration into applications.

Limitations

- Requires configuration of the Azure environment and the relevant services.
- Depends on cloud resources, model availability and service permissions.
- Advanced workflows may need additional analyzers and routing configuration.
- Cloud deployment brings cost, data governance and service-dependency considerations.

Relevance to the proposed system. Content Understanding combines classification with routing, which matches the project's main objective. The project follows the same workflow (classify, identify the category, recommend a route) but builds and compares its own models instead of relying on a managed service.

6.2.3 DocBERT: BERT for document classification

Overview. DocBERT, introduced by Adhikari et al. (2019), applies BERT to document classification [1]. BERT is a pretrained Transformer that learns contextual representations of text [7]; unlike word-frequency features, it captures relationships between words in context. The authors reported strong results on four benchmark datasets and also studied smaller models to reduce inference cost.

Advantages

- Uses contextual representations of words.
- Can be fine-tuned for different classification tasks.
- Provides a research baseline for comparing Transformers with conventional methods.

Limitations

- Needs more computation than traditional text classifiers.
- Performance depends on the dataset, training configuration and model choice.
- A new domain requires labeled examples and fine-tuning.
- Addresses classification only; a routing workflow must be built separately.

Relevance to the proposed system. DocBERT is the research foundation for the project's Transformer model. The project fine-tunes a pretrained Transformer from Hugging Face and tests whether its gains over TF-IDF models justify the extra computation, then maps predictions to routing recommendations.

6.3 Methods used in this project

Each member owns one method; this section summarizes the literature behind each.

Method	Key idea from the literature	Use in this project	Owner
TF-IDF with linear classifiers	TF-IDF weights a term by its frequency in a document and its rarity across the corpus [17]. Linear SVMs on these sparse features are strong text classifiers because text has many relevant features and is mostly linearly separable [13].	Project baseline: Logistic Regression, Linear SVM and Naive Bayes on word and character n-grams	Member 2
Ensemble methods	Random Forests average many decision trees trained on bootstrap samples [3]; XGBoost builds regularized boosted trees and handles sparse inputs efficiently [5].	Tests whether non-linear models improve on linear models with the same TF-IDF features	Member 3

Method	Key idea from the literature	Use in this project	Owner
Recurrent networks (LSTM)	LSTMs keep information over long sequences through gated memory cells [12]; pretrained word embeddings such as GloVe give them a semantic starting point [16].	Bidirectional LSTM that reads word order, which bag-of-words features ignore	Member 4
Transformers	BERT pretrains a bidirectional Transformer on large text corpora and is fine-tuned per task [7]; DistilBERT keeps most of BERT's accuracy in a smaller, faster model [18].	Fine-tuned DistilBERT or BERT-base; the expected best model	Member 5
Class imbalance	SMOTE creates synthetic minority examples by interpolating between neighbors [4]; class weighting and oversampling are simpler alternatives.	Compares class weights, oversampling and synthetic samples on per-class F1	Member 1
Confidence calibration	Modern neural networks are often overconfident; temperature scaling is a simple, effective correction [9].	Calibrated scores make the routing threshold meaningful	Member 6
Experiment tracking	MLflow records parameters, metrics, artifacts and model versions across the ML lifecycle [20].	Every run is logged; the best model is registered and served	Member 4
Benchmark data	RVL-CDIP contains 400,000 document images in 16 classes, 25,000 per class [10].	Source of OCR text for invoices, letters, memos and resumes	Member 1

6.4 Comparative analysis

Criterion	Google Cloud Document AI	Azure AI Content Understanding	DocBERT	Proposed system
Main purpose	Classification and processing	Classification and routing	Classification	Classification and routing recommendations
Approach	Managed AI	Managed AI	Pretrained	Classical ML, LSTM

Criterion	Google Cloud Document AI	Azure AI Content Understanding	DocBERT	Proposed system
	service	service	Transformer	and Transformer, compared
Custom categories	Supported	Supported	Needs task-specific fine-tuning	Defined by the project dataset
Routing	Via a broader workflow	Built-in configurable routing	Not provided	Category-to-department mapping with a confidence threshold
Model comparison	Depends on service and configuration	Depends on service and configuration	Research comparisons published	Direct comparison of six approaches on the same splits
Experiment tracking	Depends on surrounding workflow	Depends on surrounding workflow	Not part of the method	MLflow tracking and model registry
Main consideration	Cloud dependency and usage cost	Configuration and cloud dependency	Computation and adaptation	Dataset quality, model performance, implementation effort

The table compares functions, not accuracy: a fair performance comparison would require testing all solutions on the same dataset under the same conditions. Managed services provide end-to-end processing, while DocBERT focuses on the model; the proposed system combines both perspectives.

6.5 Advantages and limitations of existing approaches

The reviewed solutions show three useful principles: predefined categories make processing consistent, linking classification to routing removes a manual decision, and pretrained Transformers help when context matters.

Each approach also has costs. Managed services simplify implementation but add platform dependency and usage costs. Research models allow experimentation but need extra work to become an application. Traditional models are efficient baselines, but their effectiveness depends on the dataset. A useful solution therefore weighs accuracy together with computation, reproducibility and integration into a workflow.

6.6 Research gap and proposed improvements

The reviewed solutions each cover one part of the problem: custom classification (Document AI), classification with configurable routing (Content Understanding), or a Transformer classifier (DocBERT). None compares several model families on the same document categories inside one routing workflow, with calibrated confidence deciding when a person must step in. The project addresses this with:

1. **Comparative model evaluation:** TF-IDF with linear and ensemble classifiers, a BiLSTM and a fine-tuned Transformer, trained and tested on the same splits.
2. **Systematic evaluation:** precision, recall and F1 per class and macro-averaged, plus confusion matrices.
3. **Imbalance handling:** class weights, oversampling and synthetic samples compared on per-class F1.
4. **Calibrated routing:** calibrated confidence scores and a tuned threshold that sends uncertain documents to human review.
5. **Error analysis:** a review of misclassified and low-confidence documents to find confusing categories, ambiguous text and dataset limitations.
6. **Experiment tracking:** every configuration, metric and model version recorded in MLflow.
7. **Inference packaging:** the selected model, preprocessing and routing logic packaged as a service that returns a category, confidence and recommended department.

6.7 Conclusion

Cloud platforms show that classification and routing can be integrated into document workflows, and research such as DocBERT shows the value of pretrained Transformers. The proposed system combines a traditional baseline, deep learning models and a calibrated routing component, evaluated with consistent metrics and tracked in MLflow, to produce a reproducible prototype that can be extended later.

7. Requirements Gathering

The requirements cover the four items in the DEPI Project Documentation Guidelines: stakeholder analysis, user stories and use cases, functional requirements, and non-functional requirements. The system classifies incoming document text into six categories (application, complaint, invoice, contract, request, correspondence) and recommends where to route each document.

7.1 Stakeholder analysis

The project's direct stakeholders are the team and the lecturer/evaluators. The other groups below are the intended users of the system in a realistic deployment; they shape the requirements but are not involved in this project. Intake staff and department heads carry the most weight.

Stakeholder	Role	Main needs	Influence	Interest
Intake / mail-room staff	Primary users who submit documents	Fast, correct routing; a visible reason for each decision; an easy way to override	Medium	High
Department heads (Finance, Legal, Customer Service, etc.)	Receive routed documents and own the routing rules	Few misrouted documents; control over which category goes to which queue	High	High
Operations / process manager	Sponsor in a real deployment	Less manual inspection time; measurable gains in accuracy and processing time	High	High
Project team (six members)	Builds, trains and maintains the models	Versioned data, tracked experiments, easy retraining	Medium	High
IT / security / compliance	Hosts the system and protects the data	Access control, audit logs, no sensitive text in logs	High	Medium
Annotators / domain experts	Label the dataset (the team in this project)	Clear class definitions and edge-case rules	Low	Medium
Lecturer / evaluators	Assess the project against the DEPI guidelines	Complete deliverables on GitHub, clear evaluation and error analysis	High	High
Document senders (customers, vendors, citizens)	Indirect beneficiaries	Their documents reach the right team quickly	Low	Low

7.2 User stories and use cases

Ten user stories cover the core flow, and six use cases describe how each actor interacts with the system. Priorities use MoSCoW (Must, Should, Could).

User stories

ID	As a...	I want...	So that...	Acceptance criteria	Priority
US1	Intake clerk	to paste or upload document text and get a category	I do not read every document myself	The predicted category and recommended department are shown for valid text, with a p95 response time under 500 ms (NFR2)	Must
US2	Intake clerk	to see the destination department for each document	I know where to send it	The destination comes from the routing table and is shown next to the category	Must
US3	Intake clerk	uncertain documents sent to manual review	wrong routing is avoided	Documents with calibrated confidence below a configurable threshold are flagged for manual review instead of being treated as confirmed	Must
US4	Intake clerk	to see the top 3 categories with probabilities	I can judge borderline cases	The top-3 list is shown with scores summing to at most 1	Should
US5	Intake clerk	to submit many documents at once	I can clear a daily backlog	Batch input returns one result per document	Should
US6	Department head	to change the category-to-department mapping	routing follows our structure	A routing change takes effect without retraining or redeploying the model	Must
US7	Operations manager	a report of accuracy per category for every model	I can trust the system before relying on it	The report shows precision, recall and F1 per class plus macro averages for each model compared	Must

ID	As a...	I want...	So that...	Acceptance criteria	Priority
US8	Auditor / IT	every prediction logged with time, model version and scores	decisions can be reviewed later	Each prediction produces one log entry without storing the document text	Must
US9	ML engineer	all experiments tracked	I can compare models and reproduce results	Parameters, metrics and artifacts for every run are visible in MLflow	Must
US10	ML engineer	to retrain the model on new labeled data	accuracy keeps up with new document types	Retraining runs from one documented command and registers a new model version	Should

Use cases

ID	Use case	Actor	Main flow	Alternative / exception flow
UC1	Classify a single document	Intake clerk	1. Clerk submits text. 2. System preprocesses it. 3. Model returns category and calibrated confidence. 4. System applies the routing table and shows the destination.	Empty or oversized text: the system rejects it with a clear message.
UC2	Handle a low-confidence document	Intake clerk, reviewer	1. Confidence falls below the threshold. 2. System marks the document Manual Review. 3. The reviewer checks it and selects the category. 4. System shows the matching routing recommendation.	If the reviewer cannot determine the category, the document stays under manual review.
UC3	Classify a batch	Intake clerk	1. Clerk submits several documents. 2. System classifies each one. 3. System returns a result list.	One failed document does not stop the rest; it is reported as an error.

ID	Use case	Actor	Main flow	Alternative / exception flow
UC4	Maintain routing rules	Departme nt head	1. Head opens the routing table. 2. Head edits a category-to-department entry. 3. System saves it and uses it for new documents.	Invalid department name: the change is rejected.
UC5	Train and compare models	ML engineer	1. Engineer runs training for the linear, ensemble, LSTM and Transformer models. 2. System logs each run to MLflow. 3. Engineer compares metrics and registers the best model.	Training fails: the run is logged as failed with its error.
UC6	Review prediction logs	Auditor / IT	1. Auditor opens the logs. 2. Filters by date, model version or category. 3. Inspects scores and routing outcome.	No entries for the filter: the system shows an empty result.

7.3 Functional requirements

Twenty-one functional requirements define what the system must do; each traces back to a user story.

ID	Requirement	Priority	Story
FR1	Accept document text and return a predicted category with a confidence score	Must	US1
FR2	Map each category to a destination through a configurable routing table	Must	US2, US6
FR3	Send predictions below a tunable confidence threshold to a Manual Review queue	Must	US3
FR4	Return the top-k categories with probabilities	Should	US4
FR5	Support single and batch inference	Should	US5
FR6	Log every prediction (timestamp, model version, scores) without the document text	Must	US8
FR7	Reject empty input and input above a maximum length with a clear message	Must	US1
FR8	Apply the same preprocessing pipeline in training and inference	Must	US1

ID	Requirement	Priority	Story
FR9	Provide a labeled dataset covering the six categories, with a labeling guideline and stratified train/validation/test splits	Must	US7
FR10	Train and evaluate a TF-IDF baseline with linear classifiers (Logistic Regression, Linear SVM, Naive Bayes)	Must	US7, US9
FR11	Train and evaluate TF-IDF ensemble classifiers (Random Forest, XGBoost) on the same splits	Must	US7, US9
FR12	Train and evaluate a BiLSTM with pretrained word embeddings on the same splits	Must	US7, US9
FR13	Fine-tune a Hugging Face Transformer and compare it with the other models on the same splits	Must	US7, US9
FR14	Compare class-imbalance handling (class weights, oversampling, synthetic samples) on per-class F1	Should	US7
FR15	Calibrate confidence scores and tune the routing threshold on the validation set	Must	US3
FR16	Report precision, recall and F1 per class, macro and weighted averages, and a confusion matrix for every model	Must	US7
FR17	Produce an error analysis of misclassified and low-confidence documents	Must	US7
FR18	Log parameters, metrics, artifacts and dataset version to MLflow, and register the best model	Must	US9, US10
FR19	Package the selected model with its preprocessing and routing logic as a REST API that returns category, confidence and recommended department	Must	US1, US2
FR20	Provide a demo interface where the user pastes text and sees category, confidence and destination	Should	US1, US2, US4
FR21	Export a compressed version of the selected model (ONNX or quantization) and compare its latency	Could	US1

Out of scope for the core version: OCR of scanned documents, multi-label classification, and non-English documents. The core system takes document text as

input; text extraction from text-based PDF and DOCX files belongs to the optional upload interface, together with the Azure ML deployment.

7.4 Non-functional requirements

Ten non-functional requirements cover performance, security, usability and reliability. The numeric targets are the same as the KPIs in Section 5; they are provisional until reviewed after the baseline results (M4, 13 Nov).

ID	Category	Requirement	Measure (target)
NFR1	Accuracy	The system is evaluated on a held-out test set of real documents using precision, recall and F1	Macro F1 of 0.92 or higher for the advanced model and 0.85 or higher for the baseline; per-class precision and recall of 0.85 or higher
NFR2	Performance	Classification is fast enough for interactive use	p95 latency under 500 ms per document (up to 512 tokens) on CPU; under 200 ms for the baseline
NFR3	Scalability	Batch requests complete without failing	100 or more documents classified per minute by one service instance
NFR4	Security	Submitted text is validated, and access to the inference interface is restricted where authentication is implemented	Invalid input is rejected; no API keys or credentials in the source code or public repository
NFR5	Privacy	Sensitive content is not exposed	No document text in logs; personal data anonymized in the dataset
NFR6	Usability	A new user can classify a document without training	First classification in under 1 minute using the demo interface; 90% or more of test users succeed without help
NFR7	Reliability	One bad input does not break the service	Error rate under 1%; errors return a clear message; a batch continues after a single failure
NFR8	Maintainability	Routing rules change without code or	Routing table stored in a configuration file

ID	Category	Requirement	Measure (target)
		model changes	
NFR9	Reproducibility	Results can be repeated by another person	Fixed random seeds, pinned dependencies, dataset version logged in MLflow
NFR10	Portability	The model runs the same way on another machine	Model packaged with its preprocessing in a Docker image; the README gives steps to run the inference demonstration

8. References

1. Adhikari, A., Ram, A., Tang, R., & Lin, J. (2019). DocBERT: BERT for document classification. arXiv. https://arxiv.org/abs/1904.08398
2. Bitext. (2023). Bitext customer support LLM chatbot training dataset. Hugging Face. https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset
3. Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5-32.
4. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321-357. https://arxiv.org/abs/1106.1813
5. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD 2016*. https://arxiv.org/abs/1603.02754
6. Consumer Financial Protection Bureau. (n.d.). Consumer Complaint Database narratives archive. https://www.consumerfinance.gov/foia-requests/foia-electronic-reading-room/cfpb-consumer-complaint-database-narratives-archive/
7. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional Transformers for language understanding. *Proceedings of NAACL-HLT 2019*. https://arxiv.org/abs/1810.04805
8. Google Cloud. (n.d.). Create, use, and manage a custom document classifier. Google Cloud Documentation. https://docs.cloud.google.com/document-ai/docs/custom-classifier
9. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of ICML 2017*. https://arxiv.org/abs/1706.04599
10. Harley, A. W., Ufkes, A., & Derpanis, K. G. (2015). Evaluation of deep convolutional nets for document image classification and retrieval. *Proceedings of ICDAR 2015*. https://www.cs.cmu.edu/~aharley/rvl-cdip/
11. Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). CUAD: An expert-annotated NLP dataset for legal contract review. arXiv. https://arxiv.org/abs/2103.06268
12. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735-1780.

13. Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. *Proceedings of ECML 1998*, 137-142.
14. Klimt, B., & Yang, Y. (2004). The Enron corpus: A new dataset for email classification research. *Proceedings of ECML 2004*. Cleaned version: https://huggingface.co/datasets/corbt/enron-emails
15. Microsoft. (n.d.). Classify and route your data using Content Understanding. Microsoft Learn. https://learn.microsoft.com/azure/ai-services/content-understanding/how-to/classification-content-understanding-studio
16. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP 2014*, 1532-1543.
17. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513-523.
18. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv. https://arxiv.org/abs/1910.01108
19. Voxel51. (n.d.). High quality invoice images for OCR. https://docs.voxel51.com/dataset_zoo/datasets_hf/high_quality_invoice_images_for_ocr.html
20. Zaharia, M., Chen, A., Davidson, A., et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin, 41*(4), 39-45.